

Relatório do Projeto — Java Event Planner

SCC0504 — Versão Compacta · Java + Swing

Aluno: Fernando Lopes 12725515

1. Descrição da aplicação

O Java Event Planner é uma aplicação desktop, escrita em Java com a biblioteca gráfica Swing, para gerenciar uma agenda de eventos. A janela principal é dividida em três colunas redimensionáveis (JSplitPane): um calendário mensal, a lista de eventos do dia selecionado e uma linha do tempo no estilo Outlook, com alternância entre os modos Dia e Semana (cada evento vira um card colorido, posicionado pelo horário). O usuário pode criar, editar, excluir e buscar eventos (por palavra-chave), navegar entre os meses e voltar rapidamente ao dia atual. Cada evento possui título, data, hora, local, descrição, categoria e um tempo de antecedência de lembrete.

Ao abrir, o programa entra em tela cheia (janela maximizada), carrega os eventos salvos em arquivo e exibe, em uma janela de aviso, os lembretes cujo disparo cai nas próximas 24 horas (sem usar thread em segundo plano, conforme a versão Compacta). Como melhorias opcionais, implementamos categorias coloridas — cada dia do calendário mostra, lado a lado, as cores de todas as categorias de evento daquele dia, e as mesmas cores identificam os cards na linha do tempo — e lista de participantes (nome e e-mail) por evento.

2. Como executar

Requisito: Java 8 ou superior (testado em Java 11). A partir da raiz do projeto:

```
mkdir -p build
javac -d build $(find src -name "*.java")
java -cp build eventplanner.app.EventPlannerApp
```

Os eventos são gravados em `dados/eventos.csv` (criado automaticamente se não existir).

Detalhes no `README.md`.

3. Arquitetura

Organizamos o código em camadas, no estilo Model-View-Controller (MVC) apresentado no Cap. 13. Cada pacote tem uma responsabilidade clara, o que facilita testar, dividir o trabalho e estender a aplicação:

Pacote	Papel	Principais classes
modelo	Dados (Model)	Evento, Participante, Categoria, Antecedencia
execcao	Erros de validação	ValidacaoException e subclasses
persistencia	Ler/gravar arquivo	RepositorioEventos (interface), RepositorioEventosCSV
servico	Regras (Controller)	GerenciadorEventos
gui	Interface (View)	JanelaPrincipal, PainelCalendario, PainelEventosDia, PainelAgenda, PainelLinhaTempo, IconeCategorias, DialogoEvento, IntField
app	Inicialização	EventPlannerApp (main)

A regra de ouro: a interface gráfica conversa apenas com o `GerenciadorEventos`; este, por sua vez, fala com a persistência através de uma **interface**. A Figura 1 (ao final) mostra o diagrama de classes completo.

4. Conceitos de POO aplicados (e por quê)

Conceito	Onde aparece	Por que usamos
Encapsulamento	Atributos <code>private</code> + <code>get/set</code> em todo o modelo	Proteger o estado e centralizar validação (Cap. 3)
Herança	<code>JanelaPrincipal</code> /painéis, <code>IntField</code> , hierarquia de exceções	Reaproveitar Swing e tratar erros por tipo (Cap. 8/10)
Polimorfismo	<code>toString()/paintComponent()</code> sobrescritos; interfaces <code>Scrollable/Icon</code> e <code>RepositorioEventos</code> ; <code>catch</code> da base	Comportamento conforme o objeto (Cap. 8)
Abstração	Interface <code>RepositorioEventos</code>	Trocar CSV por outro meio sem mexer no resto (Cap. 8)
Composição	<code>Evento</code> tem <code>Vector<Participante></code> , <code>Categoria</code> , <code>Antecedencia</code>	“Tem-um” em vez de “é-um”
Exceções	<code>try/catch/finally</code> + exceções próprias + <code>JOptionPane</code>	Mensagens amigáveis, sem stack trace (Cap. 10)

Quais classes usam herança e por quê.

(1) Componentes Swing: `JanelaPrincipal` extends `JFrame`, os painéis extends `JPanel` e `IntField` extends `JTextField` — é a mesma herança que o livro usa (Cap. 4/13) para reaproveitar todo o comportamento de janela/campo e só acrescentar o nosso.

(2) Exceções: `DataInvalidaException` → `ValidacaoException` → `Exception` (Cap. 10), permitindo tratar cada erro com a mensagem certa. **Decisão consciente:** NÃO criamos subclasses de `Evento` por categoria, pois a categoria muda apenas dados (rótulo, cor), não comportamento — modelá-la como atributo (enum) é mais correto que “herança por herança”.

(3) Interfaces do Swing: `PainelLinhaTempo` implements `Scrollable` e `IconeCategorias` implements `Icon` — a 3ª forma de polimorfismo do Cap. 8 (implementar uma interface), com o desenho sobrescrito em `paintComponent/paintIcon` para renderizar a linha do tempo e as faixas de cor de cada categoria.

5. Principais desafios e soluções

- **Datas sem API nas aulas.** Os slides não cobrem manipulação de datas. Calcular “em que dia da semana cai o dia 1º” e “quantos dias tem o mês” à mão (ano bissexto, etc.) é fonte de bugs. Solução: usar `java.time` (`LocalDate/YearMonth`), isolado e comentado no código como recurso fora das aulas.
- **Lembrete de 24h sem thread.** Em vez de monitorar em segundo plano, comparamos, na inicialização, o instante do lembrete (data/hora do evento menos a antecedência) com “agora” e “agora + 24h”.
- **Persistência robusta.** Arquivo ausente → começa com agenda vazia; linha corrompida → é ignorada e as demais carregam. Usamos `try/catch` e o bloco `finally` para fechar o arquivo (Cap. 10/Class13).

- **Separar interface de lógica.** Uma camada de serviço (GerenciadorEventos) e uma interface de repositório evitam que a GUI conheça arquivos — a aula Cap. 13 mostra o ganho do MVC.
- **Validação amigável.** Entradas inválidas (título vazio, data/hora, e-mail) lançam exceções próprias capturadas pela GUI, que mostra JOptionPane — o usuário nunca vê um stack trace.
- **Cores no calendário.** Em alguns “look and feel” a cor de fundo do botão não aparece; resolvemos com setOpaque(true) e definindo o Look and Feel multiplataforma (UIManager, Cap. 13).
- **Linha do tempo com sobreposições.** Desenhar a agenda do dia/semana exigiu posicionar cada evento pela hora (Graphics2D, sobrescrevendo paintComponent) e dividir eventos que se sobrepõem em subcolunas lado a lado; o painel ainda implementa Scrollable para rolar suavemente dentro do JScrollPane, e um ícone próprio (IconeCategorias implements Icon) pinta no calendário as cores de todas as categorias de cada dia.

6. Recursos além das aulas

Para robustez, usamos alguns recursos não vistos em aula, todos **comentados no ponto de uso**: `java.time` (datas/horas), `enum` (Categoria/Antecedencia), generics em `Vector<Evento>`, `JComboBox/DefaultListModel` e `SwingUtilities.invokeLater`. Toda a base (Swing, eventos, herança, interfaces, exceções, Vector, Scanner, PrintStream) está dentro do conteúdo das aulas.

7. Conclusão

O projeto cumpre todos os requisitos da versão Compacta e exercita, de forma integrada, os principais conceitos de POO da disciplina. A arquitetura em camadas e o uso de interfaces deixam o sistema preparado para evoluir (por exemplo, lembretes em tempo real com thread ou eventos recorrentes) sem reescrever o que já existe.

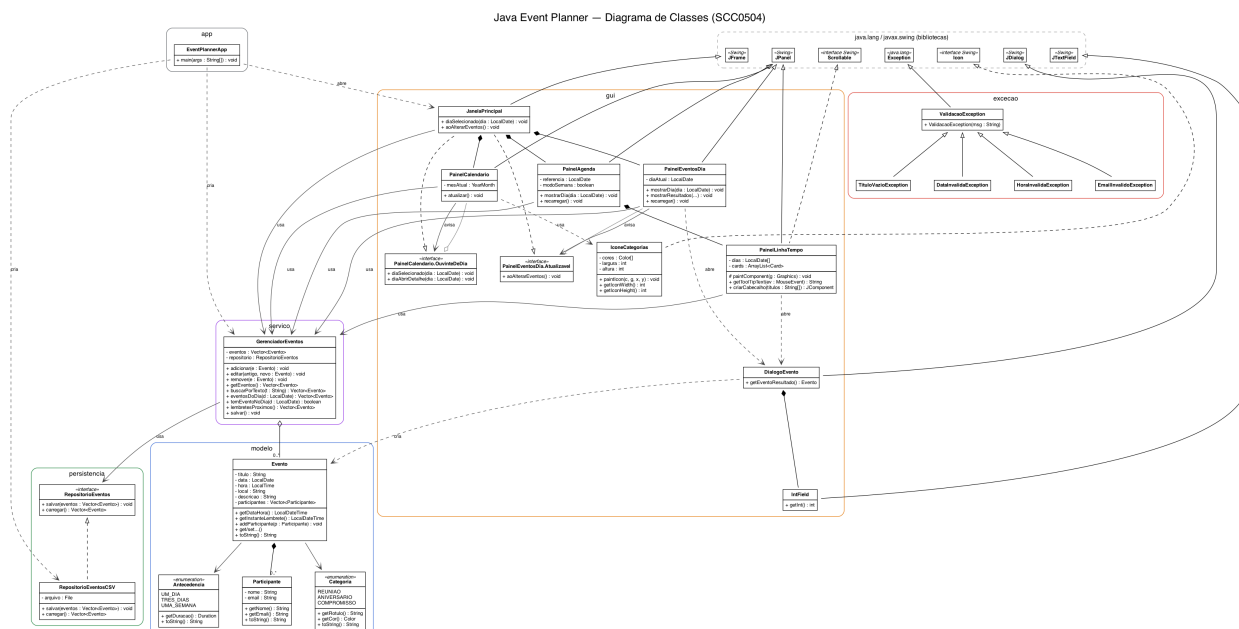


Figura 1 — Diagrama de classes (UML) do Java Event Planner.